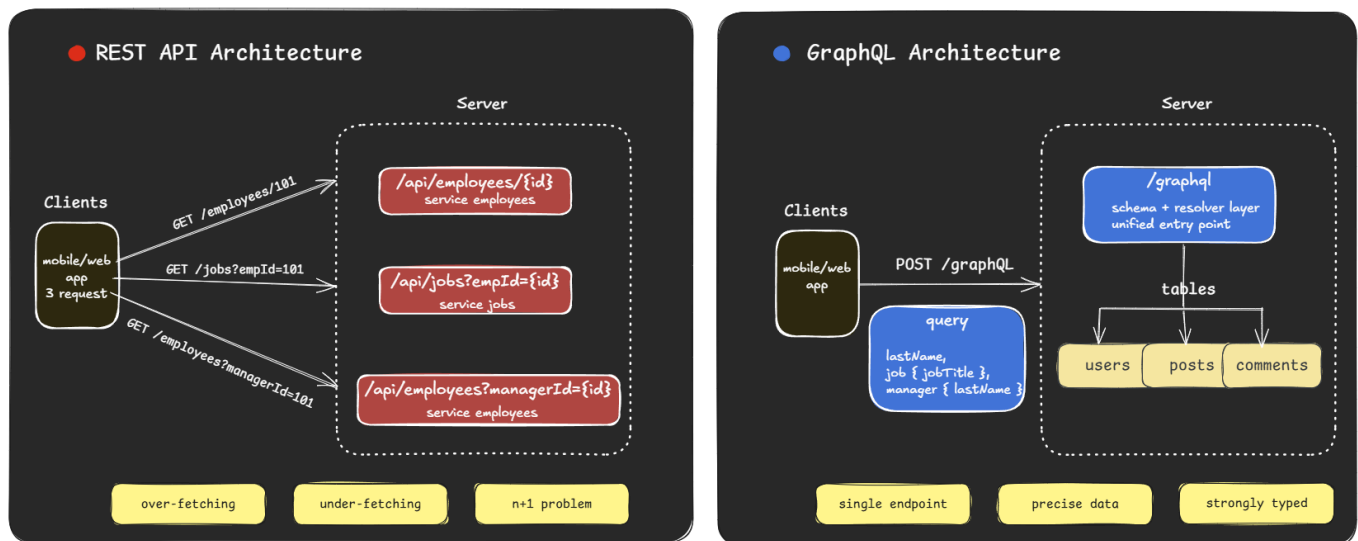


1. Konsep GraphQL

1. Diagram Architecture



2. Konsep GraphQL

GraphQL itu **bahasa query untuk API**, bukan protokol transport seperti REST. Bedanya paling mendasar:

- **REST**: bentuk response ditentukan **server**, per endpoint. Client cuma bisa terima apa adanya (GET /employees/1 selalu balikin semua field yang didefinisikan di employeeSchema).
- **GraphQL**: bentuk response ditentukan **client**, lewat satu endpoint tunggal (POST /graphql). Client bilang "aku mau field ini, relasi itu sedalam ini", server tinggal isi.

Tiga elemen intinya:

- **Schema** — kontrak tipe data (employees.typeDefs.ts), jadi single source of truth soal field apa saja yang ada dan tipenya.
- **Resolver** — fungsi yang benar-benar ngambil data per field (employees.resolvers.ts), dipanggil cuma kalau field itu diminta.
- **Satu endpoint, dua operasi** — **query** (read) dan **mutation** (write), keduanya lewat method POST yang sama.

3. Bedanya CRUD REST vs GraphQL

Secara **logic bisnis, sama persis** — resolver cuman manggil employees.service.ts yang sama dengan REST. Yang beda cuma **kulit luar**-nya:

Operasi	REST	GraphQL
Under Fetching	Selalu balikin semua field di employeeSchema, walau frontend cuma butuh nama & email buat dropdown : <pre>{ "employeeId": 1, "firstName": "...", "lastName": "...", "email": "...",</pre>	GraphQL — client minta persis yang dipakai: <pre>{ employees { items { lastName email } } }</pre> — via POST /graphql

Operasi	REST	GraphQL
	<pre>"phoneNumber": null, "hireDate": "...", "jobId": 1, "salary": 8500000, "managerId": null, ... }</pre>	
Request	REST butuh 3 request berurutan buat tampilkan 1 employee + manager + job-nya: <pre>GET /employees/1-> dapat managerId=5 jobId=2</pre> <pre>GET /employees/5-> data manager</pre> <pre>GET /jobs/2-> data job</pre>	GraphQL — 1 request, resolver yang urus di belakang layar (dan lewat DataLoader, tetap efisien walau list 20 employee): <pre>{ employee(id: 1) { lastName manager { lastName } job { jobTitle } } }</pre>
Endpoint	Di REST kita bikin endpoint baru (<code>GET /employees/:id/reports</code>) + route + controller + test baru	Di GraphQL cukup tambah field <code>directReports</code> di schema + 1 resolver (<code>employees.loaders.ts</code>) — endpoint tetap sama
Read list	<code>GET /employees?page=1&limit=20</code>	<pre>query { employees(page:1, limit:20) { items {...} } }</pre> — via <code>POST /graphql</code>
Read detail	<code>GET /employees/1</code>	<pre>query { employee(id:1) { ... } }</pre>
Create	<code>POST /employees</code> + body JSON	<pre>mutation { createEmployee(input:{...}) { ... } }</pre>
Update	<code>PATCH /employees/1</code> + body partial	<pre>mutation { updateEmployee(id:1, input:{...}) { ... } }</pre>
Delete	<code>DELETE /employees/1</code>	<pre>mutation { deleteEmployee(id:1) }</pre>
HTTP method	Beda-beda (<code>GET/POST/PATCH/DELETE</code>) — bermakna secara HTTP	Selalu POST — REST punya makna verb HTTP, GraphQL tidak (semua "operasi" ada di dalam body)
Status code error	<code>400/404/409</code> sesuai <code>ApiError</code>	Selalu 200 , error masuk ke <code>errors[]</code> dengan <code>extensions.code</code> (<code>error-mapper.ts</code> yang kita buat)
Bentuk response	Tetap (<code>schema.response</code> per route)	Dinamis, tergantung field yang diminta client

Efek praktis paling kelihatan: di REST kita bisa cek sukses/gagal cuma dari HTTP status code (`res.statusCode === 404`). Di GraphQL, HTTP status **selalu 200**, jadi wajib cek `json.errors` — itu kenapa `graphql-client.ts` yang kubuat untuk Next.js secara eksplisit cek `errors[]`, bukan `res.ok`.

5. Kelemahan GraphQL (untuk kasus di project kita)

a. N+1 problem, dan solusinya nambah kompleksitas Tanpa `DataLoader` (`employees.loaders.ts`), query dengan `job` + `manager` di 20 item bakal jadi puluhan query. Di REST kita tidak akan kena masalah ini karena bentuk response tetap.

b. Caching HTTP jadi susah REST bisa manfaatin cache HTTP standar (`GET` + `ETag/Cache-Control`, CDN, browser cache) karena URL-nya unik per resource (`/employees/1`). GraphQL semua lewat `POST /graphql` ke URL yang sama — caching di layer HTTP nggak bisa dipakai, harus caching di level aplikasi (Apollo cache normalization, dsb).

c. Tidak cocok untuk file upload Jika kita buat module `employee-files` (upload multipart) desain tetap harus pake REST, karena GraphQL secara native cuma transport JSON, upload file butuh spec tambahan (`graphql-multipart-request-spec`) yang nambah dependency & kompleksitas tanpa benefit jelas.

d. Query kompleks bisa jadi vektor serangan (query depth/complexity attack) Karena client bebas nge-nest sedalam apapun (`employee { manager { manager { manager { ... } } } }`) — self-referencing FK bikin ini secara teknis mungkin infinite), server perlu proteksi tambahan (query depth limiting, complexity scoring) yang REST tidak butuh sama sekali karena bentuk response-nya fixed.

e. Observability & monitoring lebih ribet Di REST, 1 endpoint = 1 metric/span yang jelas (`GET /employees/:id` gampang dipantau p95 latency-nya di Prometheus). Di GraphQL, 1 endpoint (`/graphql`) menangani segala macam query berbeda — perlu instrumentasi per-field/per-operation-name kalau mau observability sedetail REST (bukan cuma HTTP-level span dari `FastifyOtelInstrumentation`).

f. Learning curve & tooling tambahan Tim harus paham SDL, resolver, `DataLoader`, cara mapping error — dan sekarang ada 2 cara akses data (REST + GraphQL) yang harus dijaga tetap konsisten dengan `employees.service.ts` yang sama. Kalau tim atau konsumen API belum butuh fleksibilitas query, ini overhead murni tanpa manfaat.

Skenario: query list employee dengan `job` + `manager` tanpa `DataLoader`

```
query {
  employees(limit: 20) {
    items {
      lastName
      job { jobTitle }
      manager { lastName }
    }
  }
}
```

Kalau resolver ditulis naif (tanpa `DataLoader`, langsung query per item — ini yang biasanya orang tulis pertama kali sebelum sadar masalahnya):

```
// versi naif bruttal
Employee: {
```

```

job: (parent) => db.select().from(jobs).where(eq(jobs.jobId, parent.jobId)),
manager: (parent) => parent.managerId
      ?      db.select().from(employees).where(eq(employees.employeeId,
parent.managerId))
      : null,
}

```

Cost #1 — jumlah query bertambah sesuai jumlah rows

	findAll	query job	query manager	Total query per 1 request GraphQL
limit: 20 (default)	2 (select + count)	20	20	42 query
limit: 100 (max yang diizinkan di <code>listEmployeeQuerySchema</code>)	2	100	100	202 query

Bandingkan: REST `GET /employees?limit=100` cuma **2 query** total (select + count) — nested relation memang belum ada di REST, tapi walaupun ditambah sekalipun, jumlahnya tidak akan meledak sebesar ini karena REST biasanya join di 1 query.

Summary

GraphQL menang di fleksibilitas & efisiensi jaringan untuk data ber-relasi seperti `employees`, tapi harus dibayar di kompleksitas backend (N+1, security, observability) dan kehilangan semantik code seperti HTTP Code 200, 201, 401, 404 dan 500.

Contoh paling konkret pakai study case `employees`, plus satu detail penting: kita sudah punya `DB_POOL_MAX=10` di `.env` — ini yang bikin cost-nya kerasa nyata, bukan cuma teori.